Experiment 2: Basic Data Indexing for Retrieval-Augmented Generation (RAG)

AIM:

To build a simple Retrieval-Augmented Generation (RAG) system by loading a small text document, splitting it into chunks(e.g.,200 Tokens), generating embeddings, storing them in a vector database, retrieving the most relevant information for a user query, and generating an answer using the retrieved context.

DESCRIPTION:

Retrieval-Augmented Generation (RAG) is a technique that enhances the responses of Large Language Models (LLMs) by retrieving relevant information from an external knowledge source before generating an answer. To enable efficient retrieval, documents are converted into **embeddings** (vector representations of text) and stored in a **FAISS** vector database, which allows semantic similarity search.

In this experiment, sample documents were divided into smaller text chunks and converted into embeddings. These embeddings were indexed using FAISS to create a searchable knowledge base. When a query was given, the system retrieved the most relevant document chunks based on semantic similarity. The experiment demonstrated that data indexing enables fast and accurate information retrieval, providing the foundation for building efficient Retrieval-Augmented Generation (RAG) applications.

PROGRAM:

```
# Hide Warnings
import warnings
warnings.filterwarnings("ignore")

# Import Required Libraries
import os
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

# Hugging Face Token
os.environ["HF_TOKEN"] = "YOUR_HUGGING FACE TOKEN"

# Create Sample Documents
documents = [
    Document(page_content="Artificial Intelligence is used in healthcare for disease diagnosis and medical image analysis."),

    Document(page_content="Machine Learning is a branch of Artificial Intelligence that enables systems to learn from data."),
```

```
    Document(page_content="Deep Learning uses neural networks with multiple layers to solve
complex problems.")
]

# Split Documents into Chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0
)

chunks = text_splitter.split_documents(documents)

# Create Embedding Model
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# Create FAISS Vector Store
vectorstore = FAISS.from_documents(chunks, embeddings)

# Display Results
print("Number of Documents:", len(documents))
print("Number of Chunks:", len(chunks))
print("Vector Store Created Successfully!")

# Inspect Chunks
print("\nChunks Created:\n")

for i, chunk in enumerate(chunks, start=1):
    print("Chunk", i)
    print(chunk.page_content)
    print()
```

OUTPUT:

```
Number of Documents: 3
Number of Chunks: 3
Vector Store Created Successfully!

Chunks Created:

Chunk 1
Artificial Intelligence is used in healthcare for disease diagnosis and medical image analysis.

Chunk 2
Machine Learning is a branch of Artificial Intelligence that enables systems to learn from data.

Chunk 3
Deep Learning uses neural networks with multiple layers to solve complex problems.
```